SOP-IT-001 · VPN and DNS resolver stalls: how to tell, how to check, how to fix

Department: IT · **ID:** 001 · **Status:** draft (approved when tagged) · **Owner:** whoever runs an observer on their machine · **Last changed:** 2026-11-01

Purpose

A machine behind a VPN can lose the ability to resolve some hostnames while everything else keeps working. Bots and helpers then time out silently. This procedure says how to recognise it, prove it in a minute, and fix it, and keeps the growing list of what was found.

Scope

macOS machines running a VPN or another tunnel, on any uplink (satellite and mobile uplinks included). Each organisation's private copy keeps its own dated findings in the table below.

How to tell

- A long-running observer (a monitor, a poller) stops logging while its process is still alive.
- `gh`, `git push` or `curl` to one host hangs or says `i/o timeout`, while other sites load.
- The observer's log says `request timeout` or `getaddrinfo ENOTFOUND`.

Steps: check in one minute

1. Which resolver is first?

   ```
   scutil --dns | head -20
   ```

   You should see: `nameserver[0]` and the interface. A `100.64.x.x` address on a `utun` interface is the VPN's resolver (most VPN apps on macOS add one); `192.168.x.x` is the home router; a `100.64.x.x` address on `en0` would be the uplink's own carrier-grade NAT instead (some satellite and mobile providers use that range).
2. Does the system resolver answer for the host that is failing?

   ```
   dig +short +time=3 +tries=1 <failing-host>   # the hostname named in the error
   dig +short +time=3 +tries=1 api.github.com
   ```

   You should see: an address or alias within three seconds. Nothing, or `connection timed out`, is the stall.
3. Does a public resolver answer the same name?

   ```
   dig +short +time=3 +tries=1 @1.1.1.1 <failing-host>
   ```

   You should see: an answer. If step 2 fails and step 3 works, the VPN's resolver is the cause, not the uplink.
4. Is the VPN up and which interface carries the default route?

   ```
   netstat -rn | grep -E '^default'
   ifconfig | grep -E '^utun'   # a utun interface is a tunnel; the default route on one means
   ```

```
the VPN carries it
```

Steps: fix

1. **Give the VPN a resolver that works:** in the VPN app's settings, set a custom DNS (for example `1.1.1.1` and `1.0.0.1`), or turn off its DNS filtering (often called threat or malware protection) and reconnect. *You should see:* step 2 above answers.
2. **If it is a one-off stall:** disconnect and reconnect the VPN, then `sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder`.
3. **Make the observer independent:** give it a second endpoint to fail over to on a transport error, a way back to the first when it answers again, and an alert when it has heard nothing for ten minutes. Every observer in the fleet should do the same.
4. **Record what you saw** in the troubleshooting table below, by pull request.

Verification

Steps 2 and 3 of the check answer alike; the observer's log shows a new tick; the stall alert has not fired for an hour.

Troubleshooting (grows)

Date	Symptom	Finding	Fix
YYYY-MM-DD hh:mm → hh:mm (duration from raw logs)	an observer silent while its process is alive; `request timeout` in its error log	the VPN's resolver stopped answering one hostname; a public resolver answered	failover to a second endpoint; custom DNS in the VPN app

Related

- SOP-OPS-002 (what to do when a push hangs)
- The observer's own runbook (its failover and stall alert)
- Proposal: THEF 010 (treasury) only in that a read-only observer has the same failure mode

Change log

Date	Change	By
2026-10-30	Drafted from the Oct 29–30 outage	Alfred
2026-11-01	Adopted for The Holy Earth Foundation from the shared template: amber badges and icons, the foundation's login filled in where a page names a real address	Alfred
2026-11-01	The resolver check made generic for the public copy: a placeholder for the failing hostname, no VPN product or uplink named, the tunnel found with `ifconfig`; same steps, same fix	Alfred